Reviewer Pack — Minimal Diophantine Root Count and Frege Proof Length Correl…

Entry 96ead4e72bd7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-01 14:27:37 UTC

Minimal Diophantine Root Count and Frege Proof Length Correlation

Entry ID: `96ead4e72bd7` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-01 14:27:37 UTC

1. Conjecture

Field A (mathematical branch): Diophantine Analysis **Field B** (complexity object): Frege Proof Complexity

Statement:

> For every Frege proof system instance with n variables, the minimal number of distinct algebraic numbers required to express all clauses is linearly correlated with its proof length, such that MinRootCount(φ) = Θ(ProofLength(φ)) for Tseitin formula φ.

Rationale (proposer's reasoning):

> Diophantine analysis deals with polynomial equations and their solutions over various fields. If the number of algebraic numbers required to express all clauses in a Frege proof corresponds closely to the length of the proof, it might suggest an inherent structure in the problem that could lead to lower bounds on proof complexity.

Taxonomy category: `diophantine_analysis_frege_complexity` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `ac525e3ac4051726`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for all Tseitin formulas φ with n variables (n ≤ 40), the correlation coefficient between MinRootCount(φ) and ProofLength(φ) exceeds 0.8 over at least 100 random seeds, and there is no seed where MinRootCount(φ) > 2 * ProofLength(φ).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.70	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Diophantine Analysis" AND "Frege proof complexity" AND minimal root count" - "Tseitin formula" AND Diophantine analysis AND proof length" - "linear correlation" AND Frege proof system AND Diophantine numbers

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(A[j][i]) > abs(A[max_row][i]):
                max_row = j
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i+1, n))) / A[i][i]
    return x

def is_solution(A, b, x):
    return all(abs(sum(a*x[j] for j in range(len(x))) - b[i]) < 1e-9 for i in range(len(b)))

def min_diophantine_root_count(clauses):
    n = len(clauses)
    A = [[0]*n for _ in range(n)]
    b = [0]*n
    for i, clause in enumerate(clauses):
        for j in range(i+1, n):
            if clause[i] != 0 and clause[j] != 0:
```

```
                A[i][j] += 1
                A[j][i] += 1
            b[i] += abs(clause[i])
            b[j] += abs(clause[j])
    for i in range(n):
        A[i][i] = sum(abs(c) for c in clauses[i]) - abs(b[i])
    x = gaussian_elimination(A, b)
    return len(set(x))

def dpll_solve(clauses):
    def solve(literals):
        if not literals:
            return True
        literal = literals[0]
        if literal > 0:
            if literal in assignment or -literal not in assignment:
                assignment[literal] = True
                if solve([l for l in literals if l != literal]):
                    return True
                del assignment[literal]
            if -literal not in assignment:
                assignment[-literal] = False
                if solve([l for l in literals if l != -literal]):
                    return True
                del assignment[-literal]
        else:
            if -literal in assignment or literal not in assignment:
                assignment[-literal] = True
                if solve([l for l in literals if l != -literal]):
                    return True
                del assignment[-literal]
            if literal not in assignment:
                assignment[literal] = False
                if solve([l for l in literals if l != literal]):
                    return True
                del assignment[literal]
        return False
    n = len(clauses)
    assignment = {}
    literals = list(range(1, n+1)) + [-i for i in range(1, n+1)]
    random.shuffle(literals)
    return solve(literals)

def proof_length(clauses):
    if not clauses:
        return 0
    if all(c[0] > 0 for c in clauses):
        return 2 * proof_length([c[1:] for c in clauses]) + 1
    if any(c[0] < 0 for c in clauses):
        return 2 * proof_length([c[1:] for c in clauses if c[0] > 0]) + 1
    return 0

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    variables = set(range(1, n+1))
```

```
    clauses = []
    for _ in range(n):
        clause = [random.choice([-1, 1]) * random.choice(variables) for _ in range(random.randint(
        clauses.append(clause)
    min_root_count = min_diophantine_root_count(clauses)
    proof_len = proof_length(clauses)
    return {
        "metric_name": "MinRootCount vs ProofLength",
        "metric_value": min_root_count / proof_len,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": min_root_count <= 2 * proof_len,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [random.randint(1, 1000) for _ in range(30)]
    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        for result in results:
            if not result["conjecture_holds"]:
                print(f"RESULT: FALSIFIED counterexample=\"MinRootCount > 2 * ProofLength\" first_
                break
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e21af301.py", line 122, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e21af301.py", line 105, in run_trial
    clause = [random.choice([-1, 1]) * random.choice(variables) for _ in range(random.randint(1, n))]
                                       ^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 348, in choice
    return seq[self._randbelow(len(seq))]
           ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'set' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Investigate and fix the crash in the test code to continue testing the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15208
2	propose	ollama_remote	glm4:latest	0	0	12283
3	preregistration	ollama_remote	glm4:latest	0	0	16445
4	novelty	ollama_remote	glm4:latest	0	0	20072
5	novelty	ollama_remote	glm4:latest	0	0	16777
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21822
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21923
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22725
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22629
10	judge	ollama_remote	glm4:latest	0	0	8856

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 178740 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/96ead4e72bd7.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/96ead4e72bd7.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/96ead4e72bd7.tar.gz` (if generated)